

다중 에이전트 공유 메모리 시스템의 계층적 일관성 모델과 충돌 해결 메커니즘

멀티 에이전트 공유 메모리 시스템에서의 계층형 일관성 모델 및 충돌 해소 메커니즘 설계

YU BAOQI (위바오치)
2026년 6월

연구 배경 및 연구 동기

연구 배경 및 연구 동기

배경 및 동기

LLM 기반 멀티 에이전트 협업이 주류 패러다임으로 부상

공유 메모리는 에이전트 간 상태 동기화의 핵심 인프라

세 가지 충돌 유형이 공유 메모리의 신뢰성을 위협:

- ① 가시성 충돌 — 에이전트가 오래된 데이터를 읽음
- ② 순서성 충돌 — 과거 버전이 최신 값을 덮어씀
- ③ 의미적 충돌 — 상반된 자연어 결론이 모순됨

기존 연구 한계: 형식적 정의 부족, 전략 비교 미비, 의미 충돌 처리 미흡

연구 문제 및 연구 목표

연구 문제 및 연구 목표

5가지 연구 문제 (RQ1~RQ5)

RQ1: 세 가지 충돌 유형을 정확히 탐지하고 해결할 수 있는가?

RQ2: 계층적 일관성 모델이 단일 일관성 모델보다 우수한가?

RQ3: 타임스탬프/신뢰도/리더/Judge 네 가지 전략 중 가장 효과적인 것은?

RQ4: LLM Judge Agent가 의미적 충돌을 효과적으로 탐지할 수 있는가?

RQ5: 소거 실험을 통해 각 모듈의 독립적 기여도를 분리할 수 있는가?

연구 목표

① 세 가지 충돌 유형의 형식적 정의

② 계층적 일관성 모델 설계

③ 3단계 충돌 해결 통합

④ 8개 비교 실험 구현

⑤ 소거 실험을 통한 모듈 기여도 정량화

 방법: Design Science

 프로토타입: Python + SQLite

 LLM: DeepSeek-chat

 총: 720회 실험 실행

실험 환경 (1/3): 시스템 아키텍처 개요

실험 환경 (1/3): 시스템 아키텍처 개요

6계층 아키텍처

▶ Agent 계층

Planner / Executor / Reviewer / JudgeAgent

▶ Conflict 계층

ConflictDetector → ConflictResolver (3-Phase)

▶ Consistency 계층

Strong | Eventual | Hierarchical

▶ Strategy 계층

Timestamp | TrustScore | Leader | Judge

▶ Memory 계층

SQLite (WAL): Working Memory + Long-Term Memory

▶ Metrics 계층

MetricsCollector → Reporter → CSV

데이터 흐름

- ① 시나리오 생성 → 충돌 시나리오
- ② Agent 쓰기 → 공유 메모리
- ③ 충돌 감지 → 버전 비교
- ④ 구조 필터링 → 규칙 선별
- ⑤ Judge → LLM 의미 판단
- ⑥ 최종 결정 → 메모리에 다시 쓰기
- ⑦ 지표 수집 → 통계 분석

기술 스택

Python 3.12 · NumPy

SQLite (WAL mode)

DeepSeek-chat API

OpenAI SDK (호환)

Windows 11 환경

GitHub 오픈소스 저장소

실험 규모

8개 비교 그룹

3가지 충돌 시나리오

30회 반복/시나리오

—
= 720회 실행

—
3개 Agent:

Planner

Executor

Reviewer

—
랜덤 시드: 42

실험 환경 (2/3): 코드 구조와 핵심 모듈

실험 환경 (2/3): 코드 구조 및 핵심 모듈

핵심 클래스 및 함수

prototype/ 프로젝트 구조

```
prototype/
├── config.py      # 설정 상수 및 API 키
├── main.py        # 진입점: 전체 실험 실행
├── agents/
│   ├── base.py   # BaseAgent (읽기/쓰기 및 신뢰도 추적)
│   ├── roles.py  # Planner/Executor/Reviewer
│   └── judge.py   # JudgeAgent (LLM 의미 판정)
├── memory/
│   ├── models.py # MemoryEntry, ConflictRecord
│   └── store.py  # SQLite 공유 메모리 저장소 (WAL)
├── conflict/
│   ├── detector.py # 구조적 충돌 감지기
│   └── resolver.py # 3단계 충돌 해결 파이프라인
├── consistency/
│   ├── strong.py  # 강한 일관성 (잠금 메커니즘)
│   ├── eventual.py # 최종 일관성 (비차단)
│   └── hierarchical.py # 계층 일관성 (라우팅)
├── strategies/
│   ├── timestamp.py # 타임스탬프 우선 전략
│   ├── trust_score.py # 신뢰도 우선 전략
│   └── leader.py    # 리더 중재 전략
├── experiments/
│   ├── runner.py  # ExperimentRunner 메인 루프
│   ├── scenarios.py # 3가지 시나리오 + 15개 모순 문장 쌍
│   ├── groups.py  # 8개 비교 그룹 ResolverConfig
│   ├── metrics.py # MetricsCollector 지표 집계
│   └── reporter.py # CSV 내보내기 및 결과 출력
```

▶ SharedMemoryStore

SQLite CRUD, 버전 관리, 충돌 로그 [store.py]

▶ ConflictDetector

버전 비교 탐지 visibility/ordering [detector.py]

▶ ConflictResolver.resolve()

4단계: 탐지→필터→Judge→결정 [resolver.py]

▶ ResolverConfig

부울 플래그로 8개 구성 스위치 제어 [resolver.py:L19]

▶ JudgeAgent.judge()

DeepSeek API → JSON 파싱 [judge.py]

▶ ScenarioGenerator

3가지 시나리오 × 15개 모순 문장 쌍 [scenarios.py]

▶ HierarchicalConsistency

memory_type에 따라 자동 라우팅 [hierarchical.py]

▶ ExperimentRunner.run_all()

메인 루프: 8개 그룹 × 3개 시나리오 × 30라운드 [runner.py]

▶ build_groups()

8개의 ResolverConfig 반환 [groups.py]

실험 환경 (3/3): 8개 비교 그룹 및 평가 지표

실험 환경 (3/3): 8개 비교군 및 평가 지표

G1 · Baseline

Last-Write-Wins만 적용

G2 · StrongOnly

강한 일관성만, 전략 없음

G3 · Timestamp

타임스탬프 우선 + PreFilter

G4 · TrustScore

신뢰도 우선 + PreFilter

G5 · Leader

리더 중재 + PreFilter

G6 · Ablation1 (NoJudge)

계층형+전략, Judge 제외

G7 · Ablation2 (NoPreFilter)

계층형+전략+Judge, PreFilter 제외

G8 · FullProposed ★

제안 방식: 모든 구성 요소 통합



계층 일관성 모델 설계

계층형 일관성 모델 설계



작업 메모리 | Working Memory

키: task_plan, task_status, current_step

일관성: 강한 일관성 (Strong Consistency)

메커니즘: Lock-based Serialization

→ 모든 읽기/쓰기 직렬 실행

목적: 실시간 협업 정보의 엄격한 정합성 보장



장기 메모리 | Long-term Memory

키: knowledge_base, decision_log, feedback_history

일관성: 최종 일관성 (Eventual Consistency)

메커니즘: Non-blocking Write

→ 충돌 시 Last-Write-Wins 수렴

목적: 축적형 지식의 확장성과 유연성 확보

설계 철학 및 구현

작업 메모리 = 잠금 기반 직렬화

장기 메모리 = 비차단 쓰기

- ▶ HierarchicalConsistency 클래스: memory_type에 따라 자동으로 해당 프로토콜로 라우팅
- ▶ upgrade_to_long_term(): 검증된 작업 메모리 항목을 안전하게 장기 메모리로 승격

3단계 충돌 해결 메커니즘

3단계 충돌 해소 메커니즘

단계1: 충돌 탐지

ConflictDetector가 AgentState 버전 비교

- Visibility: $\text{seen_ver} < \text{current_ver}$
- Ordering: $0 < \text{seen_ver} < \text{current_ver}$

→ 출력: 충돌 유형 (visibility/ordering)

단계2: 구조적 사전 필터링

세 가지 전략 (택일):

- 타임스탬프 우선: 가장 최신 쓰기 값 선택
- 신뢰도 우선: 높은 신뢰도 Agent 값 선택
- 리더 중재: Reviewer 규칙 판단

→ 구조적 충돌 빠르게 해결

→ 불필요한 LLM 호출 감소

단계3: Judge Agent 의미 판단

DeepSeek-chat API 호출

- JSON 형식: {conflict, resolution, reasoning}
- 의미 모순 탐지 + 최적 값 선택

→ 자연어 값에만 호출 (>40자/구두점 포함)

→ needs_semantic_check() 정밀 필터링

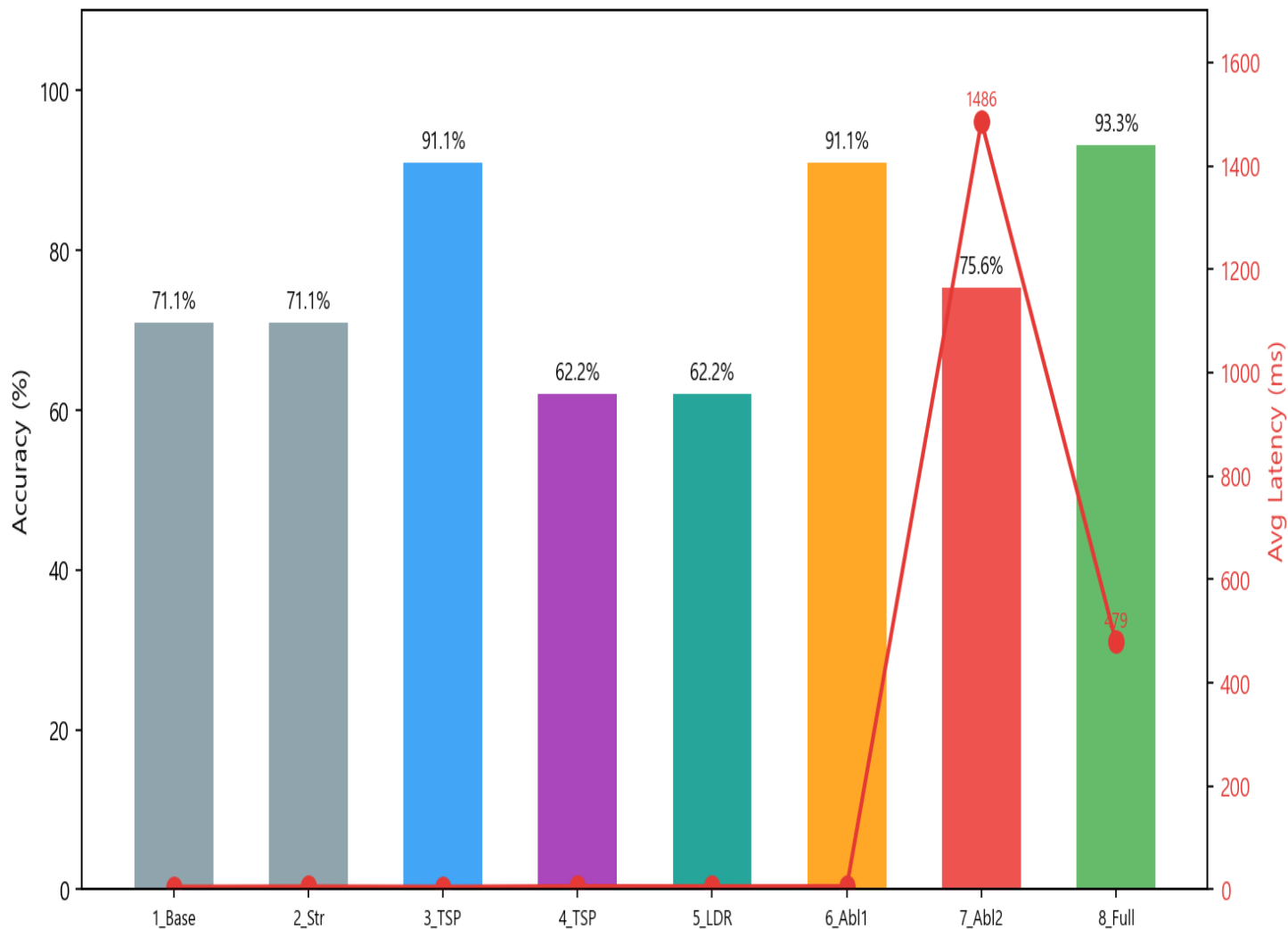
통합 의사결정 (ConflictResolver.resolve)

```
def resolve(key, value_a, value_b):  
1) detect(agent_state, key) → 구조 충돌 유형 (visibility / ordering)  
2) if pre_filter: apply(timestamp | trust_score | leader) → 구조 필터링  
3) if judge and needs_semantic_check(v_a, v_b): judge(key, v_a, v_b) → LLM 의미 판단  
4) final = judge_result or structural_result or last_write_wins → 최종 값 반환
```

실험 결과 (1/3): 종합 성능 비교

실험 결과 (1/3): 종합 성능 비교

Figure 1. Comparison Group Overall Accuracy and Average Latency



핵심 발견

Baseline (71.1%) 대비 +22.2%p

✓ **일관성 유지율: 100% (G3-G8)**

Baseline & StrongOnly: 0%

Ablation2 (1485.8ms) 대비 3.1×↓

구조적 충돌에서 가장 효과적

⚠ **TrustScore / Leader: 62.2%**

_x000B_의미 충돌은 26.7% — 단일 전략 한계

충돌 탐지는 일관성 확보의 필요조건

실험 결과 (2/3): 시나리오 분석 & 소거 실험

실험 결과 (2/3): 시나리오별 분석 및 소거 실험

Figure 2. Accuracy by Scenario per Comparison Group

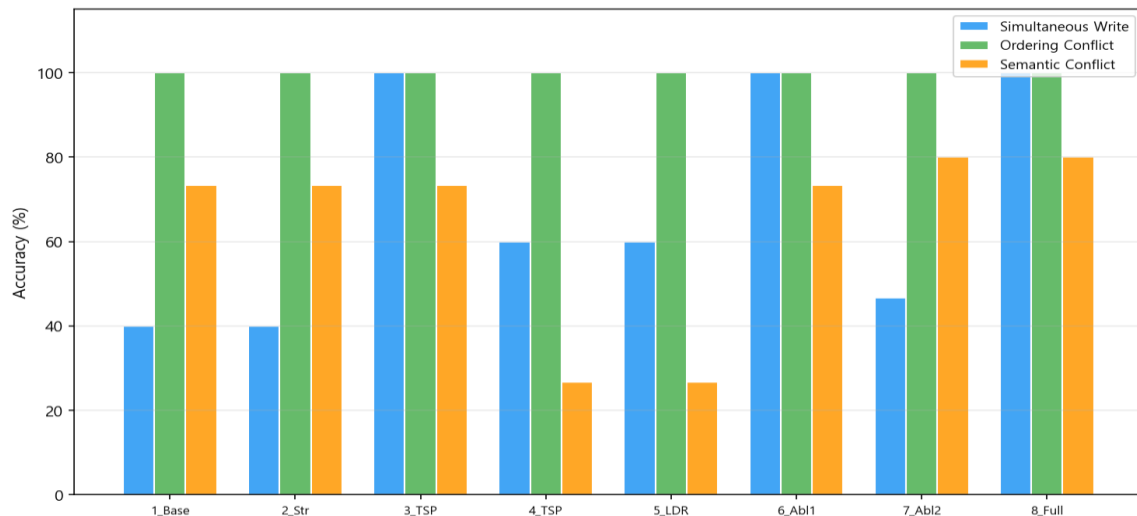


그림2: 시나리오별 정확도

Figure 3. Ablation Study: Performance Change vs FullProposed

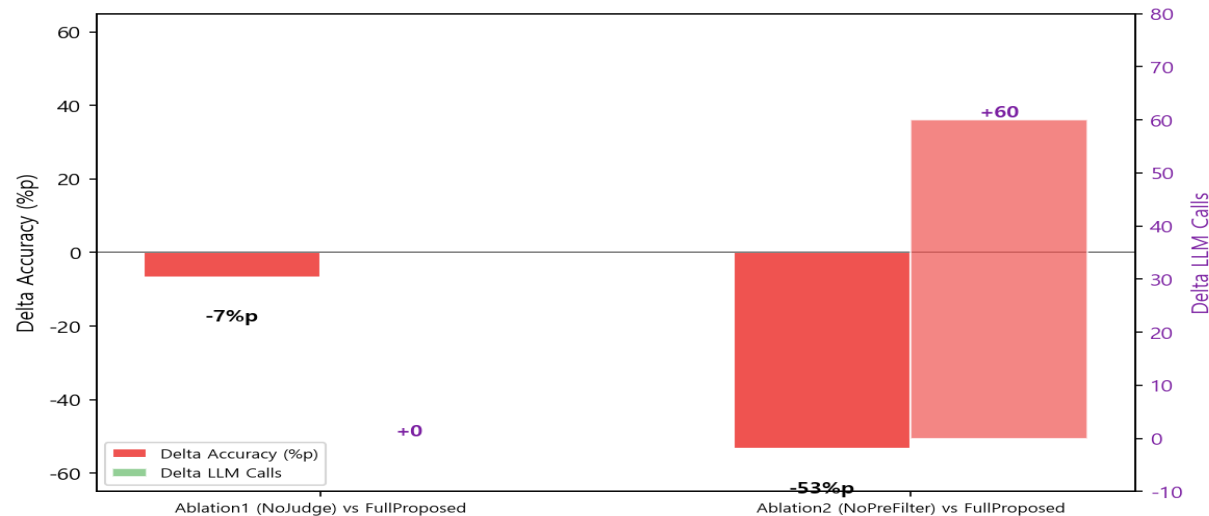


그림3: 소거 실험 결과

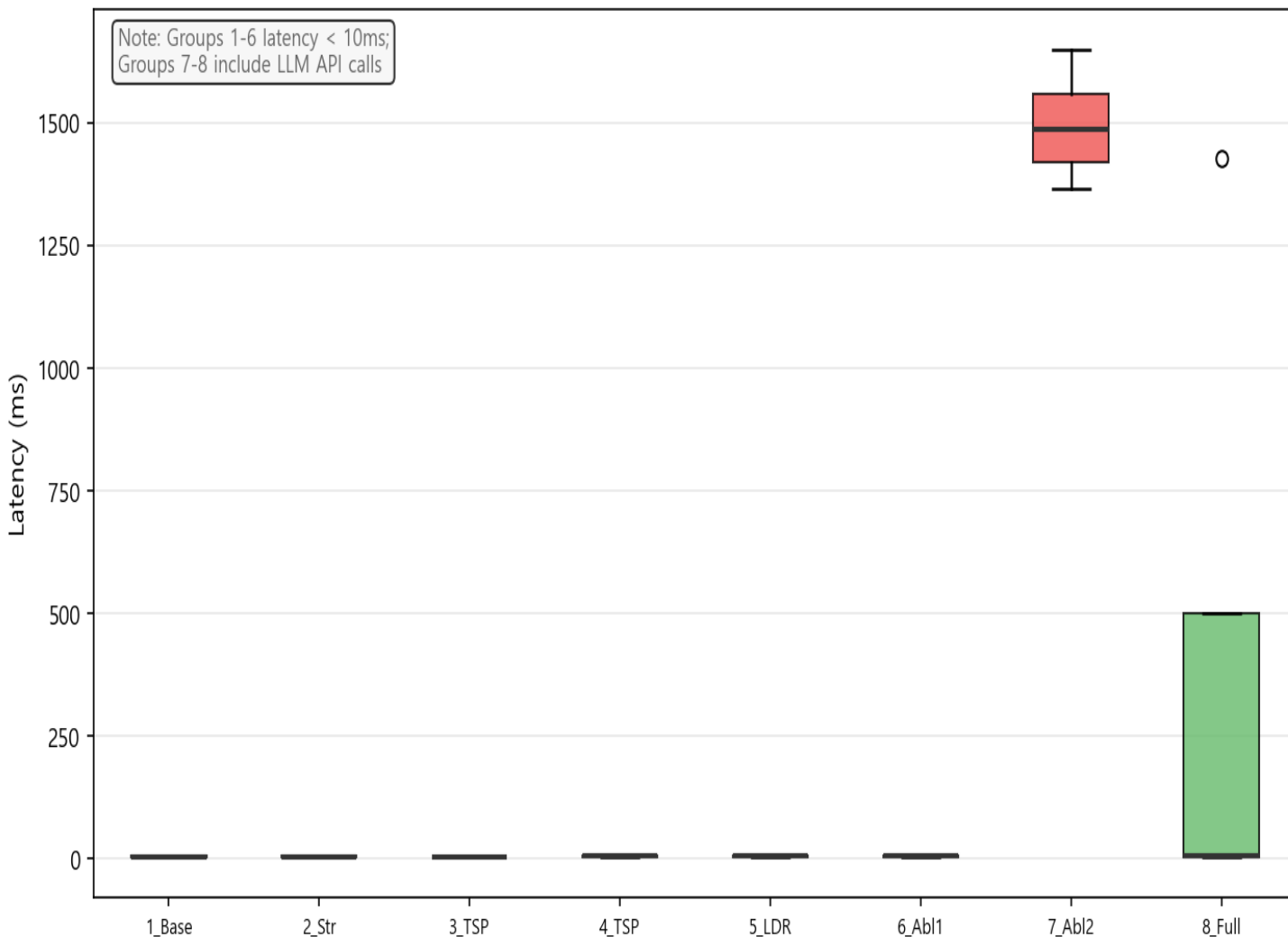
소거 실험 주요 발견

- Judge Agent 제거 시 의미적 충돌 정확도 6.67%p 하락 → Judge Agent의 의미 충돌 기여도 입증
- 사전 필터링 제거 시 LLM 호출 3배 증가, 동시 쓰기 정확도 46.7%로 급락
- 사전 필터링 + Judge Agent 통합이 최적의 정확도-효율성 균형점 달성

실험 결과 (3/3): 지연 분석

실험 결과 (3/3): 지연시간 분석

Figure 4. Latency Distribution by Comparison Group (Box Plot, N=90)



지연시간 분석

● 저지연 그룹 (G1-G6)

모든 시나리오 < 10ms

순수 구조적 처리, LLM 호출 없음

■ 고지연 그룹 (G7 Ablation2)

1300-1700ms (모든 시나리오)

실행 시마다 LLM 호출 (90회)

● 중간 그룹 (G8 FullProposed)

구조적 충돌: < 10ms

의미 충돌: LLM 호출 시 증가

평균 지연: 478.7ms

G7 Ablation2보다 3.1배 낮음

💡 사전 필터의 실용적 가치

불필요한 LLM 호출 차단

지연 감소 · 변동성 감소 · 비용 절감

결론 및 기여

결론 및 기여

5대 핵심 기여

가시성·순서성·의미적 충돌의 형식적 정의 및 탐지 조건 구체화

작업 메모리(강한 일관성) + 장기 메모리(최종 일관성)의 계층형 설계

충돌 탐지 → 구조적 사전 필터링 → LLM Judge Agent 의 통합 파이프라인

8개 비교군과 소거 실험을 통한 모듈별 기여도 정량적 검증

FullProposed: 정확도 93.3% (+22.2%p), 일관성 100%, 지연 3.1배 개선

한 줄 요약

계층형 일관성 모델과

구조적-의미적 통합

충돌 해소 메커니즘을 통해

멀티 에이전트 공유 메모리의

신뢰성과 효율성을

동시에 향상시켰다.

—

계층적 일관성 모델을 통해

구조-의미 통일 충돌 해결 메커니즘

동시에 다중 에이전트

공유 메모리 시스템의

신뢰성과 효율성을 향상시킵니다.